

Topic 4 — Unsupervised Learning

Intelligent Product Recommendation & Market Basket Pattern Discovery System

Difficulty: Easy → Medium

Domain: E-Commerce / Retail / Data Mining

ML Type: Unsupervised Learning — Association Rule Mining

1. Project Description

Develop a full-stack system that analyzes historical customer transactions to discover **which products are frequently purchased together** and uses these patterns to generate product recommendations.

For example, after discovering:

Customers who purchase **Laptop + Mouse** frequently also purchase **Laptop Bag**.

The system can recommend:

None

Customer is viewing:
Laptop

Recommended:
→ Wireless Mouse
→ Laptop Bag
→ Keyboard

The core ML/data-mining component will use **Association Rule Mining**, particularly:

- Apriori
- FP-Growth

This makes the project different from conventional classification and clustering projects.

2. Problem Statement

Retail companies generate large numbers of transactions but may not know which products have strong purchasing relationships.

The system should automatically discover:

None
Product A
+
Product B
↓
Product C

and quantify the strength of these relationships.

The application should provide:

- Product association discovery
- Product recommendations
- Frequently purchased itemsets
- Association-rule analysis
- Interactive dashboards
- Single-product recommendation
- Multi-product recommendation
- CSV batch analysis

3. Dataset

A transaction dataset can contain:

None
Transaction_ID
Customer_ID

```
Product_ID
Product_Name
Category
Quantity
Date
```

Example:

```
None
T001 | C101 | P15 | Laptop      | Electronics
T001 | C101 | P28 | Mouse       | Accessories
T001 | C101 | P31 | Laptop Bag  | Accessories

T002 | C102 | P15 | Laptop      | Electronics
T002 | C102 | P42 | Keyboard    | Accessories
```

The transaction ID is used to determine which products were purchased together.

4. Machine Learning / Data Mining Pipeline

```
None
Transaction Dataset
    ↓
Data Cleaning
    ↓
Transaction Formation
    ↓
One-Hot Encoding
    ↓
Frequent Itemset Mining
    ↓
Association Rule Generation
    ↓
```

Rule Filtering



Rule Ranking



Recommendation Engine

5. Algorithms

Algorithm 1 — Apriori

Students should implement Apriori to discover frequent itemsets.

Example:

None

{Laptop}

{Mouse}

{Laptop, Mouse}

{Laptop, Laptop Bag}

{Laptop, Mouse, Laptop Bag}

Algorithm 2 — FP-Growth

Implement FP-Growth and compare its performance with Apriori.

Compare:

Algorithm	Runtime	Itemset s	Rule s
Apriori	—	—	—
FP-Growth	—	—	—

This introduces an important engineering concept:

6. Association Rules

Generate rules such as:

None

Laptop → Mouse

or:

None

Laptop + Mouse → Laptop Bag

Each rule should contain:

Support

How frequently the combination occurs.

Confidence

How frequently the consequent occurs when the antecedent occurs.

Lift

How strongly associated the products are compared with random occurrence.

Example:

None

Rule:

Laptop → Laptop Bag

Support: 8.2%

Confidence: 72.5%

Lift: 2.41

7. Rule Filtering

Do not display thousands of meaningless rules.

Allow the user to configure:

None

Minimum Support

Minimum Confidence

Minimum Lift

Example:

None

Support ≥ 0.02

Confidence ≥ 0.40

Lift ≥ 1.20

The system should dynamically regenerate/filter the rules.

8. Recommendation Engine

Create a recommendation layer on top of the association rules.

Example

User selects:

None

Laptop

System searches:

None

Laptop → ?

and returns:

None

Recommended Products

- | | |
|-------------------|------------|
| 1. Laptop Bag | Lift: 2.41 |
| 2. Wireless Mouse | Lift: 2.18 |
| 3. Keyboard | Lift: 1.94 |

Multi-Product Recommendation

If the user selects:

None

Laptop

Mouse

the system can search for rules such as:

None

Laptop + Mouse → Laptop Bag

and return:

None

Recommended:

Laptop Bag

9. Model Extraction

Even though Association Rule Mining is different from traditional supervised ML, the trained/discovered artifacts should still be persisted.

Example:

```
None
models/
|
├─ association_rules.pkl
├─ frequent_itemsets.pkl
├─ product_encoder.pkl
└─ product_mapping.pkl
```

The recommendation backend should load these files.

It should **not rerun Apriori/FP-Growth every time a customer asks for a recommendation.**

10. Python Backend

Use **FastAPI**.

POST /recommend

Input:

```
JSON
{
  "products": [
    "Laptop"
  ]
}
```

Output:

```
JSON
{
```



```
  "recommendations": [
    {
      "product": "Laptop Bag",
      "confidence": 0.725,
      "lift": 2.41
    },
    {
      "product": "Wireless Mouse",
      "confidence": 0.681,
      "lift": 2.18
    }
  ]
}
```

POST /recommend-multiple

Input:

```
JSON
{
  "products": [
    "Laptop",
    "Mouse"
  ]
}
```

Output:

```
None
Laptop Bag
Keyboard
USB Hub
```

GET /rules

Return filtered association rules.

GET /products

Return available products.

GET /analytics

Return:

- Total transactions
 - Total products
 - Number of frequent itemsets
 - Number of association rules
 - Top-selling combinations
-

GET /health

Backend health check.

11. Streamlit Frontend

Page 1 — Dashboard

Display:

None	
TOTAL TRANSACTIONS	50,000
TOTAL PRODUCTS	420
FREQUENT ITEMSETS	1,245

ASSOCIATION RULES

3,820

Also show:

- Most frequently purchased products
- Most common product combinations
- Top association rules

12. Page 2 — Product Recommendation

User selects:

None

Product:

[Laptop ▼]

Click:

Generate Recommendations

Output:

None

RECOMMENDED PRODUCTS

Laptop Bag

Confidence: 72.5%

Lift: 2.41

Wireless Mouse

Confidence: 68.1%

| Lift: 2.18 |

13. Page 3 — Market Basket Analyzer

Allow the user to select multiple products:

None

Selected Products:

✓ Laptop

✓ Mouse

The system finds:

None

Laptop + Mouse

↓

Laptop Bag

↓

Keyboard

Display the relevant association rules.

14. Page 4 — Association Rule Explorer

Allow filtering by:

None

Minimum Support

[—●—]

Minimum Confidence



Minimum Lift



Display:

Antecedent	Consequent	Support	Confidence	Lift
Laptop	Bag	8.2%	72.5%	2.41
Mouse	Keyboard	6.7%	65.2%	1.94

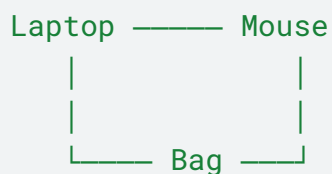
15. Page 5 — Analytics

Visualizations:

- Product frequency
- Top product combinations
- Support vs confidence
- Confidence vs lift
- Category combinations
- Transaction trends

A **network graph** can optionally show:

None



where edge strength represents association strength.

16. Page 6 — Algorithm Comparison

Compare:

None
Apriori
VS
FP-Growth

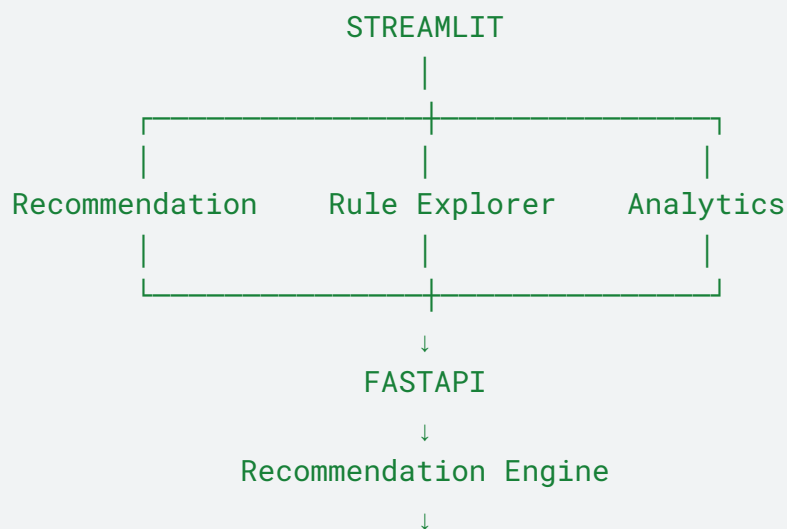
Metrics:

- Runtime
- Number of frequent itemsets
- Number of rules
- Memory usage, where measurable

This makes the project more appropriate for an engineering final-year project.

17. Architecture

None



```
association_rules.pkl
      ↓
Rule Filtering / Ranking
      ↓
Recommendations
```

18. Project Structure

```
None
market-basket-recommendation/
|
├── data/
│   └── transactions.csv
|
├── notebooks/
│   ├── 01_EDA.ipynb
│   └── 02_Association_Rules.ipynb
|
├── training/
│   ├── preprocess.py
│   ├── apriori_model.py
│   ├── fp_growth_model.py
│   ├── evaluate.py
│   └── rule_generation.py
|
├── models/
│   ├── association_rules.pkl
│   ├── frequent_itemsets.pkl
│   ├── product_encoder.pkl
│   └── product_mapping.pkl
|
└── backend/
    ├── app.py
    └── recommender.py
```

```
|   └─ schemas.py
|
| └─ frontend/
|   └─ streamlit_app.py
|
| └─ utils/
|   ├── analytics.py
|   ├── visualization.py
|   └─ rule_filter.py
|
| └─ outputs/
|   └─ generated_rules.csv
|
| └─ requirements.txt
| └─ README.md
| └─ .gitignore
```

19. Required Technologies

Data Mining

- Pandas
- NumPy
- MLxtend
- Scikit-learn

Algorithms

- Apriori
- FP-Growth

Backend

- FastAPI
- Uvicorn
- Joblib

Frontend

- Streamlit
 - Plotly
 - Matplotlib
-

20. requirements.txt

```
None
numpy
pandas
scikit-learn
mlxtend
matplotlib
plotly
joblib
fastapi
uvicorn
streamlit
```

21. Expected Final-Year Deliverables

Data Mining

```
None
EDA
↓
Transaction Processing
↓
Apriori
↓
FP-Growth
↓
```

Frequent Itemsets

↓

Association Rules

↓

Rule Evaluation

Model/Data Artifacts

None

association_rules.pkl

frequent_itemsets.pkl

product_encoder.pkl

product_mapping.pkl

Backend

None

FastAPI

REST APIs

Recommendation Engine

Rule Filtering

Batch Analysis

Frontend

None

Streamlit

Dashboard

Recommendation System

Market Basket Analyzer

Rule Explorer

Analytics

Algorithm Comparison
